InfraPilot

Developer Build Guide · v1.0

Multi-Cloud DevOps · DevSecOps · SRE Automation Platform

Stack	Java 17 · Spring Boot 3 · Vaadin 24 · PostgreSQL 15
CI/CD	Jenkins · GitHub Actions · Terraform · Docker
Security	Gitleaks · Checkov · Trivy · Spring Security
Ops	Prometheus · Grafana · AWS CloudWatch · Infracost
Cloud	AWS (Phase 1) → Azure → GCP
Author	Senior Dev Review — Revised Architecture
Status	Client-Ready Build Specification

This document is the complete specification for building InfraPilot — from architecture to folder structure, API design, database schema, CI/CD pipeline, security, and phase-by-phase implementation plan. Each section includes ChatGPT/Cursor prompts you can use directly.

Table of Contents

01 Why Java + Vaadin — Learning Decision Explained

02 Senior Dev Verdict — What Changed & Why

03 Revised Architecture (Complete)

04 Technology Stack — Final Choices

05 Complete Folder & Repository Structure

06 Database Schema — PostgreSQL

07 API Design — Spring Boot Endpoints

08 CI/CD Pipeline — Jenkins Stages

09 Security Implementation — DevSecOps

10 Infrastructure — Terraform AWS

11 Observability — Prometheus + Grafana

12 Phase-by-Phase Build Plan (7 Phases)

13 MVP Definition — What to Demo First

14 Client Sales Pitch + Pricing Model

15 ChatGPT Prompts — Build Each Component

■ 01. Why Java + Vaadin — Learning Decision

HONEST TAKE: Vaadin is NOT the industry standard in 2025. But for LEARNING Java full-stack — it is a valid and smart choice. Yeh decision production client ke liye nahi, tere growth ke liye hai. Aur yeh document mein clearly document karo.

Learning Goal	Java frontend + backend dono sikhna — Vaadin ek hi language mein dono deta hai
Market Reality	React/Angular dominate frontend market. Vaadin <2% job postings mein hai
When Vaadin is OK	Learning, enterprise Java shops, rapid prototyping, small internal tools
When Vaadin fails	Large team, React devs available, modern UI/UX required, public-facing SaaS
Your Strategy	Build Phase 1-2 with Vaadin, learn core concepts. Phase 3+ mein React layer add karo optionally
Resume Note	Document karo 'Built full-stack Java app with Vaadin 24' — yeh skill dikhata hai
Client Note	Client ko explain karo: Vaadin = Java-based, maintainable by Java devs, no JS overhead

DECISION LOCKED: We keep Vaadin 24 with Java 17. But we add proper component structure, routing, and responsive design — not just slap some Vaadin components together.

■ 02. Senior Dev Verdict — What Changed & Why

Yeh 8 major changes hain jo ek senior developer immediately karta before client delivery:

CHANGE 1	**Add Message Queue**	CRITICAL
	Deployment requests are ASYNC. Jenkins trigger, Terraform apply — sab time leta hai. Without a queue, your Spring Boot API blocks. Add: AWS SQS (free tier) or RabbitMQ (self-hosted).	
CHANGE 2	**Add Redis Cache**	HIGH
	Provider capability checks, user sessions, deployment status polling — ye sab bar bar DB hit karte hain. Redis add karo, DB load 60% kam hoga. Use: Spring Cache + Redis starter.	
CHANGE 3	**Add Infracost to Pipeline**	HIGH
	Client puchega 'kitna cost aayega?' Jenkins pipeline mein Infracost step add karo. Terraform plan ke baad automatically cost estimate show ho. Tool: infracost breakdown --path=. command in Jenkinsfile.	
CHANGE 4	**Multi-tenancy Design Clarify**	CRITICAL
	Document vague hai. Clearly define: PostgreSQL Row-Level Security per org_id, Terraform workspaces per tenant, Jenkins folders per org, S3 prefix isolation per tenant. Bina iske data leak possible.	
CHANGE 5	**Add WebSocket Notifications**	MEDIUM
	User ko pipeline status real-time chahiye. HTTP polling mat karo — Vaadin has built-in Push (WebSocket) support. Use @Push annotation in Vaadin + Spring WebSocket config.	
CHANGE 6	**Add API Rate Limiting**	HIGH
	SaaS product hai — ek tenant 1000 requests maar sakta hai per minute. Add Bucket4j (Java rate limiting library) in Spring Boot filter layer. Simple, no extra infra needed.	
CHANGE 7	**Add OpenTelemetry Tracing**	MEDIUM
	Prometheus metrics good hai — but distributed tracing missing hai. Ek request Spring Boot se Jenkins se Terraform tak trace karna mushkil hoga. Add OpenTelemetry Java agent + Jaeger or Tempo.	
CHANGE 8	**Secrets Management — Properly**	CRITICAL
	Document vaguely mentions 'dont store secrets in plain text'. Properly implement: HashiCorp Vault or AWS Secrets Manager. Spring Vault starter se easy integration. Jenkins credentials plugin for pipeline.	

■■ 03. Revised Complete Architecture

This is the CORRECTED architecture with all 8 changes applied. Every layer has a clear responsibility. No vague components.

LAYER 1 — FRONTEND	**Vaadin 24 + Java 17**
	MainLayout (AppLayout), Views (LoginView, DashboardView, DeployView, MonitorView, AuditView, SettingsView). Push enabled for real-time pipeline status. Vaadin Router for navigation. Responsive Grid layout.
LAYER 2 — API GATEWAY	**Spring Boot + Bucket4j**
	All requests enter via /api/v1/** prefix. JWT validation filter. Rate limiting: 100 req/min per tenant (Bucket4j in-memory or Redis). Request logging via MDC with tenant_id, user_id, request_id.
LAYER 3 — BUSINESS LOGIC	**Spring Boot Services**
	AuthService (JWT + BCrypt), UserService, OrgService, ProjectService, DeploymentService (creates request + queues), PipelineService (Jenkins API), ProviderService (capability checks), AuditService (immutable logs).
LAYER 4 — MESSAGE QUEUE	**AWS SQS / RabbitMQ**
	deployment-requests queue: new deployment triggers Jenkins asynchronously. pipeline-events queue: Jenkins callbacks update deployment status. dead-letter queue: failed messages for debugging. No blocking API calls.
LAYER 5 — CI/CD ENGINE	**Jenkins + Shared Library**
	Stages: Checkout → Unit Tests → Lint → Secret Scan (Gitleaks) → IaC Scan (Checkov) → Image Scan (Trivy) → Infracost Estimate → Terraform Plan → Approval Gate → Terraform Apply → App Deploy → Smoke Tests → Notify (SQS callback).
LAYER 6 — INFRASTRUCTURE	**Terraform + AWS**
	modules/: vpc, ec2, rds, ecr, iam, s3, cloudwatch. State: S3 bucket per workspace + DynamoDB lock table. Workspaces: {org_id}-{project_id}-{env}. Infracost: cost estimate in Terraform plan output.
LAYER 7 — DATA LAYER	**PostgreSQL 15 + Redis 7**
	PostgreSQL: Multi-tenant with org_id RLS policies on all tables. Redis: Session cache (TTL 1hr), deployment status cache (TTL 5min), provider capability cache (TTL 24hr), rate limit counters.
LAYER 8 — SECRETS	**HashiCorp Vault / AWS Secrets Manager**
	Cloud credentials per tenant: stored in Vault, accessed via AppRole. Jenkins: credentials plugin + Vault plugin for pipeline secrets. App secrets: Spring Cloud Vault starter. No secrets in DB or code.
LAYER 9 — OBSERVABILITY	**Prometheus + Grafana + OTel**

	Metrics: Spring Actuator → Prometheus scrape. Dashboards: Deployment frequency, DORA metrics, SLO compliance, cost trends per tenant, security scan results over time. Tracing: OpenTelemetry Java agent → Jaeger/Tempo. Alerts: Alertmanager → Slack/Email.

■■ 04. Technology Stack — Final Choices

Layer	Technology	Version	Why This Choice	Change?
Frontend	Java + Vaadin	17 + 24.x	Learning Java full-stack. AppLayout, Grid, Charts built-in	Kept (learning)
Backend	Spring Boot	3.2.x	Industry standard Java backend. Auto-config, starters	Kept ■
Auth	Spring Security + JWT	6.x	RBAC, stateless auth, filter chain customizable	Kept ■
Rate Limit	Bucket4j	8.x	Java-native, Spring integration, Redis-backed	NEW ■
Database	PostgreSQL	15.x	ACID, RLS for multi-tenancy, JSONB for flex data	Kept ■
Cache	Redis	7.x	Session, status, capability caching. Spring Cache abstraction	NEW ■
Queue	AWS SQS	—	Async deployment triggers. Free tier generous. Simple API	NEW ■
Secrets	HashiCorp Vault	1.15.x	Industry standard. Spring Cloud Vault starter.	NEW ■
CI/CD	Jenkins	2.4xx LTS	Self-hosted control. Shared library for reuse. Groovy DSL	Kept ■
IaC	Terraform	1.6.x	HCL modules, state management, provider ecosystem	Kept ■
Cost	Infracost	0.10.x	Terraform plan → cost estimate. Free for open source	NEW ■
Containers	Docker	24.x	Standard image packaging. Compose for local dev	Kept ■
Config Mgmt	Ansible	2.15.x	Limited use — host config, operational tasks only	Kept (limited)
Monitoring	Prometheus + Grafana	2.48 + 10.x	Standard observability stack. Alertmanager for alerts	Kept ■
Tracing	OpenTelemetry	1.x	Distributed tracing. Java agent. Vendor-neutral	NEW ■
Security Scan	Gitleaks + Checkov + Trivy	Latest	Secret + IaC + Container scanning. Jenkins integration	Kept ■
Cloud	AWS (Phase 1)	—	EC2, RDS, S3, ECR, SQS, Secrets Manager, CloudWatch	Kept ■

■ 05. Complete Repository & Folder Structure

This is the PRODUCTION-GRADE folder structure. Every folder has a purpose. Follow this exactly — it will make code reviews, onboarding, and CI/CD much easier.

```
infrapilot/ ■■■ frontend/ # Vaadin UI (Java 17) ■ ■■■
src/main/java/com/infrapilot/ui/ ■ ■ ■■■ MainLayout.java # AppLayout — nav, sidebar,
header ■ ■ ■■■ views/ ■ ■ ■ ■■■ LoginView.java # Auth — JWT login form ■ ■ ■ ■■■
DashboardView.java # Home — deployment stats, recent activity ■ ■ ■ ■■■ DeployView.java
# New deployment wizard (multi-step) ■ ■ ■ ■■■ PipelineView.java # Jenkins pipeline
status + logs ■ ■ ■ ■■■ InfraView.java # Terraform plan viewer ■ ■ ■ ■■■
SecurityView.java # Scan results (Gitleaks/Checkov/Trivy) ■ ■ ■ ■■■ MonitorView.java #
Grafana embeds + health checks ■ ■ ■ ■■■ AuditView.java # Immutable audit log viewer ■
■ ■ ■■■ CostView.java # Infracost breakdown per deployment ■ ■ ■ ■■■ SettingsView.java
# Org, users, cloud credentials ■ ■ ■■■ components/ # Reusable Vaadin components ■ ■ ■
■■■ DeploymentCard.java # Status card with pipeline link ■ ■ ■ ■■■ ScanResultBadge.java
# Pass/Fail/Warning badge ■ ■ ■ ■■■ CostSummary.java # Cost estimate display ■ ■ ■ ■■■
HealthIndicator.java # Green/Red/Yellow health dot ■ ■ ■■■ security/ ■ ■ ■ ■■■
VaadinSecurityConfig.java # Vaadin + Spring Security bridge ■ ■ ■■■ push/ ■ ■ ■■■
PipelineEventBroadcaster.java # WebSocket push for live updates ■ ■■■
src/main/resources/ ■ ■■■ application.yml ■ ■■■ themes/infrapilot/ # Custom Vaadin theme
(CSS variables) ■ ■■■ backend/ # Spring Boot 3 REST API ■ ■■■
src/main/java/com/infrapilot/ ■ ■ ■■■ api/ ■ ■ ■ ■■■ v1/ ■ ■ ■ ■ ■■■
AuthController.java # POST /auth/login, /auth/refresh ■ ■ ■ ■ ■■■
DeploymentController.java # CRUD /deployments ■ ■ ■ ■ ■■■ ProjectController.java # CRUD
/projects ■ ■ ■ ■ ■■■ OrgController.java # CRUD /organizations ■ ■ ■ ■ ■■■
PipelineController.java # GET /pipelines/{id}/status ■ ■ ■ ■ ■■■
ProviderController.java # GET /providers/capabilities ■ ■ ■ ■ ■■■
SecurityController.java # GET /scans/{deploymentId} ■ ■ ■ ■ ■■■ CostController.java #
GET /cost/{deploymentId} ■ ■ ■ ■ ■■■ AuditController.java # GET /audit?org=&from;=&to;=
■ ■ ■■■ domain/ ■ ■ ■ ■■■ model/ # JPA entities ■ ■ ■ ■ ■■■ User.java ■ ■ ■ ■ ■■■
Organization.java ■ ■ ■ ■ ■■■ Project.java ■ ■ ■ ■ ■■■ Deployment.java ■ ■ ■ ■ ■■■
PipelineRun.java ■ ■ ■ ■ ■■■ ScanResult.java ■ ■ ■ ■ ■■■ AuditEvent.java ■ ■ ■ ■ ■■■
CostEstimate.java ■ ■ ■ ■■■ repository/ # Spring Data JPA repos ■ ■ ■ ■■■ service/ #
Business logic ■ ■ ■ ■■■ AuthService.java ■ ■ ■ ■■■ DeploymentService.java # Core
orchestration ■ ■ ■ ■■■ JenkinsService.java # Jenkins REST API client ■ ■ ■ ■■■
TerraformService.java # Plan/Apply management ■ ■ ■ ■■■ ProviderService.java #
AWS/Azure/GCP capability checks ■ ■ ■ ■■■ VaultService.java # Secret management ■ ■ ■
■■■ SQSService.java # Queue publish/consume ■ ■ ■ ■■■ AuditService.java # Immutable
audit logging ■ ■ ■■■ config/ ■ ■ ■ ■■■ SecurityConfig.java # JWT filter chain + RBAC
■ ■ ■ ■■■ RedisConfig.java # Cache configuration ■ ■ ■ ■■■ SQSConfig.java # AWS SQS
client beans ■ ■ ■ ■■■ VaultConfig.java # HashiCorp Vault config ■ ■ ■ ■■■
RateLimitConfig.java # Bucket4j filter setup ■ ■ ■ ■■■ OpenTelemetryConfig.java # OTel
tracer bean ■ ■ ■■■ security/ ■ ■ ■ ■■■ JwtFilter.java ■ ■ ■ ■■■ RateLimitFilter.java
■ ■ ■ ■■■ TenantContextHolder.java # ThreadLocal org_id ■ ■ ■■■ exception/ ■ ■ ■■■
GlobalExceptionHandler.java ■ ■ ■■■ ErrorResponse.java ■ ■■■ src/main/resources/ ■
■■■ application.yml ■ ■■■ application-dev.yml ■ ■■■ application-prod.yml ■ ■■■
db/migration/ # Flyway SQL migrations ■ ■■■ V1__init_schema.sql ■ ■■■
V2__add_rls_policies.sql ■ ■■■ V3__add_audit_table.sql ■ ■■■ V4__add_cost_table.sql ■
■■■ infrastructure/ ■ ■■■ modules/ # Shared Terraform conventions ■ ■ ■■■ tagging/ #
Mandatory resource tagging module ■ ■■■ aws/ ■ ■ ■■■ modules/ ■ ■ ■ ■■■ vpc/ # VPC,
subnets, IGW, route tables ■ ■ ■ ■■■ ec2/ # Auto Scaling Group, Launch Template ■ ■ ■
■■■ rds/ # RDS PostgreSQL, subnet group, SG ■ ■ ■ ■■■ ecr/ # Elastic Container Registry
■ ■ ■ ■■■ alb/ # Application Load Balancer ■ ■ ■ ■■■ s3/ # State bucket, artifacts
bucket ■ ■ ■ ■■■ iam/ # Least-privilege roles per service ■ ■ ■ ■■■ sqs/ # Deployment
+ events queues ■ ■ ■ ■■■ cloudwatch/ # Log groups, metric filters, alarms ■ ■ ■ ■■■
secrets-manager/ # Secret rotation config ■ ■ ■■■ templates/ ■ ■ ■ ■■■
fullstack-standard/ # VPC + EC2 + RDS + ALB ■ ■ ■ ■■■ container-standard/ # VPC + ECS +
RDS + ECR ■ ■ ■ ■■■ serverless-basic/ # Lambda + API GW + RDS Proxy ■ ■ ■■■ backend.tf
```

```
# Remote state S3 + DynamoDB lock ■ ■■■ azure/ # Phase 5 placeholder ■ ■■■ gcp/ # Phase
6 placeholder ■ ■■■ pipelines/ ■ ■■■ Jenkinsfile # Main pipeline entrypoint ■ ■■■
shared-library/ ■ ■■■ vars/ ■ ■ ■■■ runTests.groovy ■ ■ ■■■ runSecurityScan.groovy #
Gitleaks + Checkov + Trivy ■ ■ ■■■ runInfracost.groovy # Cost estimate step ■ ■ ■■■
runTerraform.groovy # Plan + Approval + Apply ■ ■ ■■■ deployApp.groovy ■ ■ ■■■
runSmokeTests.groovy ■ ■ ■■■ notifyStatus.groovy # SQS callback to API ■ ■■■
src/com/infrapilot/ ■ ■■■ PipelineUtils.groovy ■ ■■■ monitoring/ ■ ■■■ prometheus/ ■ ■
■■■ prometheus.yml ■ ■ ■■■ rules/ ■ ■ ■■■ deployment-alerts.yml ■ ■ ■■■
slo-alerts.yml ■ ■ ■■■ cost-alerts.yml ■ ■■■ grafana/ ■ ■ ■■■ dashboards/ ■ ■ ■■■
deployments.json ■ ■ ■■■ slo-compliance.json ■ ■ ■■■ security-scans.json ■ ■ ■■■
cost-trends.json ■ ■■■ alertmanager/ ■ ■■■ alertmanager.yml # Slack + email routing ■
■■■ security/ ■ ■■■ policies/ ■ ■ ■■■ checkov-baseline.yaml ■ ■ ■■■
severity-policy.json # CRITICAL=block, HIGH=warn, etc. ■ ■■■ scanner-config/ ■ ■ ■■■
gitleaks.toml ■ ■ ■■■ trivy.yaml ■ ■ ■■■ sonarqube.properties ■ ■■■ vault/ ■ ■■■
policies/ ■ ■■■ app-policy.hcl ■ ■■■ pipeline-policy.hcl ■ ■■■ tests/ ■ ■■■ unit/ #
JUnit 5 + Mockito ■ ■■■ integration/ # @SpringBootTest + Testcontainers ■ ■■■
acceptance/ # Cucumber BDD scenarios ■ ■■■ infra/ # Terratest for Terraform modules ■
■■■ docs/ ■ ■■■ architecture.md ■ ■■■ security-model.md ■ ■■■ sre-runbook.md ■ ■■■
disaster-recovery.md ■ ■■■ tenant-isolation.md # NEW — multi-tenant design doc ■ ■■■
cost-governance.md # NEW — cost controls doc ■ ■■■ api-spec.yaml # OpenAPI 3.0 spec ■
■■■ scripts/ ■ ■■■ setup-local.sh # Docker Compose + local dev setup ■ ■■■
seed-dev-data.sql # Dev test data ■ ■■■ dr-test.sh # Disaster recovery drill script ■
■■■ docker-compose.yml # Local dev: PG + Redis + Vault + Prometheus ■■■
docker-compose.prod.yml # Production-ready compose ■■■ README.md # Quick start +
architecture overview
```

■■ 06. Database Schema — PostgreSQL with RLS

All tables have org_id for multi-tenancy. Row-Level Security (RLS) is enabled on every table — a tenant can NEVER see another tenant's data. Flyway handles all migrations.

```
-- ■■■ ORGANIZATIONS
■■■
```

```
scan_results ENABLE ROW LEVEL SECURITY; CREATE POLICY scans_tenant ON scan_results USING
(org_id = current_setting('app.org_id')::UUID); -- ■■■ COST ESTIMATES
■■■
```

■ 07. API Design — Spring Boot REST Endpoints

*All endpoints under /api/v1/**. JWT required except /auth/**. Every response includes request_id for tracing. Versioned from day 1.*

Method	Endpoint	Role Required	Description
POST	/auth/login	Public	Login — returns JWT access + refresh tokens
POST	/auth/refresh	Public	Refresh JWT using refresh token
POST	/auth/logout	Any	Invalidate refresh token
GET	/organizations/me	Any	Get current user's organization
PUT	/organizations/{id}	admin	Update org settings
GET	/organizations/{id}/members	admin	List org members
POST	/organizations/{id}/invite	admin	Invite user to org
GET	/projects	Any	List projects in current org
POST	/projects	devops|admin	Create new project
GET	/projects/{id}	Any	Get project details
PUT	/projects/{id}	devops|admin	Update project config
DELETE	/projects/{id}	admin	Delete project (requires approval)
GET	/deployments	Any	List deployments (paginated, filtered)
POST	/deployments	developer+	Submit new deployment request
GET	/deployments/{id}	Any	Get deployment details + status
POST	/deployments/{id}/approve	devops|admin	Approve pending deployment
POST	/deployments/{id}/reject	devops|admin	Reject deployment
POST	/deployments/{id}/rollback	devops|admin	Trigger rollback
DELETE	/deployments/{id}	admin	Cancel pending deployment
GET	/pipelines/{deploymentId}/status	Any	Get pipeline run status + stages
GET	/pipelines/{deploymentId}/logs	devops+	Get Jenkins build logs
GET	/pipelines/{deploymentId}/plan	Any	Get Terraform plan output
GET	/scans/{deploymentId}	Any	Get all scan results for deployment
GET	/scans/{deploymentId}/{type}	Any	Get specific scan (secrets|iac|container)
GET	/cost/{deploymentId}	Any	Get Infracost estimate for deployment
GET	/cost/org/summary	admin	Monthly cost summary for org
GET	/providers	Any	List supported cloud providers
GET	/providers/{cloud}/capabilities	Any	Get provider capability map
GET	/providers/{cloud}/regions	Any	List supported regions
GET	/audit	admin	Get audit log (paginated, date-filtered)
GET	/audit/export	admin	Export audit log as CSV
GET	/health	Public	App health check (no auth)
GET	/metrics	Internal	Prometheus metrics endpoint

■ 08. CI/CD Pipeline — Jenkins Stages

```
// Jenkinsfile — infrapilot/pipelines/Jenkinsfile // Uses Shared Library:
@Library('infrapilot-shared-lib') @Library('infrapilot-shared-lib') _ pipeline { agent {
label 'infrapilot-agent' } // Isolated agent per org parameters { string(name:
'DEPLOYMENT_ID', defaultValue: '', description: 'InfraPilot deployment UUID') string(name:
'ORG_ID', defaultValue: '', description: 'Tenant org ID') string(name: 'CLOUD_PROVIDER',
defaultValue: 'aws', description: 'aws|azure|gcp') string(name: 'ENVIRONMENT',
defaultValue: 'dev', description: 'dev|staging|prod') string(name: 'TEMPLATE',
defaultValue: 'fullstack-standard') string(name: 'REPO_URL', defaultValue: '',
description: 'Application repo') string(name: 'COMMIT_SHA', defaultValue: 'HEAD') }
environment { DEPLOYMENT_ID = "${params.DEPLOYMENT_ID}" INFRAPILOT_API =
"https://api.infrapilot.internal" AWS_REGION = "ap-south-1" INFRACOST_API_KEY =
credentials('infracost-api-key') } stages { // ■■ STAGE 1: CHECKOUT
■■■
```

```
'**/terraform-plan.txt,**/scan-*.json', fingerprint: true cleanWs() } } }
```

■ 09. Security Implementation — DevSecOps

Control	Tool/Method	Where	Action on Failure
Secret Detection	Gitleaks	Pipeline Stage 3	ABORT — revoke + rotate immediately
IaC Misconfiguration	Checkov + baseline	Pipeline Stage 4	CRITICAL=ABORT, HIGH=WARN
Container Vulnerabilities	Trivy	Pipeline Stage 5	CRITICAL CVE=ABORT
Code Quality	SonarQube (optional)	Pipeline Stage 2.5	Quality gate failure=WARN
JWT Security	Spring Security 6	Every API call	401 Unauthorized
Rate Limiting	Bucket4j	API Gateway layer	429 Too Many Requests
Multi-tenancy	PostgreSQL RLS	Every DB query	Data never visible cross-tenant
Secret Storage	HashiCorp Vault	App + Jenkins	No plain-text secrets anywhere
Terraform State	S3 + DynamoDB lock	Per-tenant workspace	Encrypted + access controlled
Audit Trail	PostgreSQL append-only	Every sensitive action	Immutable, exportable
Production Gate	Jenkins input() + RBAC	Pipeline Stage 8	Cannot deploy without approval
Credentials in Pipeline	Jenkins credentials plugin	All Jenkinsfile secrets	No secrets in code

Severity Policy — What Blocks the Pipeline:

Severity	Action	Example
CRITICAL	Pipeline ABORTS — deployment cancelled	Exposed AWS key, CVSS 9.0+ CVE, hardcoded password
HIGH	Warning shown — devops must acknowledge	Overly permissive IAM role, HIGH CVE in image
MEDIUM	Logged — shown in dashboard	Unused security group rule, medium CVE
LOW / INFO	Informational only	Formatting, minor code smell

■ 10. Infrastructure — Terraform AWS (Phase 1)

Every resource MUST have mandatory tags. Terraform workspaces isolate tenants. State in S3 + DynamoDB lock. Infracost estimates run before every apply.

```
# infrastructure/aws/templates/fullstack-standard/main.tf terraform { required_version =
">= 1.6.0" required_providers { aws = { source = "hashicorp/aws", version = "~> 5.0" } }
backend "s3" { # Values injected by pipeline per workspace # bucket =
"infrapilot-tfstate-${org_id}" # key = "${workspace}/terraform.tfstate" # region =
"ap-south-1" # dynamodb_table = "infrapilot-tfstate-lock" # encrypt = true } } variable
"org_id" { type = string } variable "project_id" { type = string } variable "environment"
{ type = string } # dev|staging|prod variable "deployment_id"{ type = string } variable
"app_name" { type = string } variable "region" { type = string default = "ap-south-1" }
variable "instance_type"{ type = string default = "t3.micro" } variable "db_instance" {
type = string default = "db.t3.micro" } # Mandatory tags — enforced by tagging module
locals { common_tags = { org_id = var.org_id project_id = var.project_id environment =
var.environment deployment_id = var.deployment_id managed_by = "infrapilot"
created_at = timestamp() } } # Instance size whitelist — prevent oversized dev resources
variable "allowed_instance_types" { default = { dev = ["t3.micro", "t3.small"] staging =
["t3.small", "t3.medium"] prod = ["t3.medium", "t3.large", "m5.large"] } } # Validation —
fail fast on invalid config resource "null_resource" "validate_instance" { lifecycle {
precondition { condition = contains( var.allowed_instance_types[var.environment],
var.instance_type ) error_message = "Instance type ${var.instance_type} not allowed in
${var.environment}" } } } # ■■■ MODULES
■■■
```

■ 11. Observability — Prometheus + Grafana + Alerts

Dashboard	Key Metrics Shown	Alert Rule
Deployment Overview	Total today, success rate, avg duration, failed count	Success rate < 80% for 1h
Pipeline Health	Stage durations, scan pass rate, approval wait time	Stage timeout > 30min
Security Posture	Critical findings trend, scan pass rate per tool	Critical findings > 0
Cost Trends	Monthly spend per org, cost per deployment, budget %	Budget > 80% consumed
SLO Compliance	99.9% uptime SLO, error budget remaining, burn rate	Error budget burn rate > 2x
Infrastructure Health	EC2 CPU/mem, RDS connections, ALB latency/5xx	5xx rate > 1% for 5min
Platform Health	API latency p99, queue depth, DB pool usage	API p99 > 2s for 5min

Key Prometheus Metrics to Expose (Spring Actuator):

```
# application.yml — Prometheus metrics config management: endpoints.web.exposure.include: health,info,metrics,prometheus metrics: tags: application: infrapilot org_id: ${app.current-org-id:unknown} # Custom metrics to register in DeploymentService.java: # Counter: infrapilot_deployments_total{status, environment, provider} # Histogram: infrapilot_deployment_duration_seconds{template, environment} # Counter: infrapilot_scan_results_total{tool, severity, status} # Gauge: infrapilot_active_pipelines{provider, environment} # Counter: infrapilot_cost_estimate_usd_total{org_id, environment} # Counter: infrapilot_api_requests_total{endpoint, method, status}
```

■■ 12. Phase-by-Phase Build Plan

Do NOT start Phase 2 before Phase 1 is working end-to-end. Each phase has a demo scenario — show it before moving on. Estimated solo developer timelines. Team of 2-3 = roughly half the time.

Phase 0	**Foundation Setup**	Week 1-2
	• Git repo with folder structure from Section 05	
	• Docker Compose: PostgreSQL + Redis + Vault (dev mode) + Prometheus	
	• Spring Boot skeleton: health endpoint, Flyway V1 migration runs	
	• Vaadin shell: LoginView + MainLayout with sidebar navigation	
	• README with local dev setup instructions	
	• DEMO: App starts, login page loads, DB connected	
Phase 1	**AWS Infrastructure Core**	Week 3-5
	• Terraform modules: VPC, EC2, RDS, ALB written and tested	
	• S3 state bucket + DynamoDB lock table created manually	
	• Terraform workspace per environment working	
	• Deploy a sample Java app manually using your modules	
	• Tagging module enforced — no tag = plan fails	
	• DEMO: terraform apply deploys working Java app on AWS	
Phase 2	**Jenkins Pipeline**	Week 6-8
	• Jenkins running in Docker with Java + Docker + Terraform agents	
	• Shared library: runTests, runSecurityScan, runTerraform, deployApp	
	• Gitleaks + Checkov integration — results as JSON artifacts	
	• Infracost step — cost shown in console output	
	• Terraform plan → manual approval → apply flow working	
	• DEMO: Push code → Jenkins runs → security scans → plan → deploy	
Phase 3	**Spring Boot API + SQS**	Week 9-11
	• All REST endpoints from Section 07 implemented	
	• JWT auth + Spring Security RBAC	
	• AWS SQS: deployment-requests queue → triggers Jenkins job	
	• Jenkins SQS callback → updates deployment status in DB	
	• Bucket4j rate limiting active on all endpoints	
	• DEMO: POST /deployments triggers Jenkins, status updates via API	
Phase 4	**Vaadin Dashboard**	Week 12-14

	• All Vaadin views connected to API (not mock data)	
	• DeployView: multi-step wizard → submits to API	
	• PipelineView: real Jenkins status with @Push WebSocket updates	
	• SecurityView: scan results displayed with severity badges	
	• CostView: Infracost breakdown rendered as cost table	
	• DEMO: Full flow from dashboard login to deployed app visible	

Phase 5	**Security + Vault + Audit**	Week 15-16
	• HashiCorp Vault: cloud credentials per tenant	
	• PostgreSQL RLS: enable and test tenant isolation	
	• Audit log: all sensitive actions recorded immutably	
	• Trivy image scanning added to pipeline	
	• Security severity policy enforced (CRITICAL=abort)	
	• DEMO: Inject secret in code → pipeline aborts. Cross-tenant access denied.	

Phase 6	**Observability + SLOs + Polish**	Week 17-18
	• Grafana dashboards: 4 dashboards from Section 11	
	• Alertmanager: Slack notifications for critical alerts	
	• OpenTelemetry: distributed tracing Java agent configured	
	• SLO definitions documented and monitored	
	• DR runbook written + backup validation tested	
	• DEMO: Simulate failed deployment → alert fires → rollback executed	

■ 13. MVP Definition — What to Demo First

MVP = Phases 0 + 1 + 2 (partial). This is what you show a client in Week 8-10. One complete flow, no half-built features.

Step	What Happens	Who Does It	Dashboard Shows
1	User signs in to InfraPilot	Developer (demo user)	Dashboard with empty deployment list
2	Creates new project: selects AWS, names it, picks dev env, fullstack-standard template	Developer	Project card created
3	Submits deployment request with repo URL + commit SHA	Developer	Status: QUEUED
4	Jenkins pipeline triggers automatically via SQS	System	Status: RUNNING — stages visible
5	Secret scan runs — no findings	Jenkins (Gitleaks)	Stage: SECRET SCAN ■
6	IaC scan runs — 2 medium findings shown	Jenkins (Checkov)	Stage: IaC SCAN ■■ 2 medium
7	Infracost shows estimate: ~$23/month for this stack	Jenkins (Infracost)	Cost: $23.40/month
8	Terraform plan shown for review	Jenkins	Plan: +12 resources, $23.40/mo
9	DevOps approves from dashboard	DevOps (approver)	Status: APPLYING
10	Terraform applies, app deployed to EC2 + RDS	Jenkins + Terraform	Status: DEPLOYED
11	Smoke test passes, endpoint URL shown	Jenkins	Endpoint: http://alb.aws.com ■
12	Grafana dashboard shows app health metrics	Prometheus	CPU: 12%, Latency: 45ms

■ 14. Client Sales Pitch + Pricing Model

Yeh section tab use karo jab tum is product ko kisi company ko sell karna chahte ho. Realistic positioning — not overselling.

One-Line Pitch:

"InfraPilot is an internal developer platform that lets your team deploy applications to AWS in one click — with automated security scanning, cost estimates, and full audit trail. No more manual Terraform, no more Jenkins config per project, no more deployment surprises."

Target Customers:

Customer Type	Problem They Have	Your Solution
Startups (5-50 devs)	No dedicated DevOps — devs waste time on infra	Self-service deploys in 1 click
Mid-size SaaS companies	Manual deployments, inconsistent envs, no security gates	Automated pipeline with built-in scans
Dev agencies	Multiple clients on multiple clouds — chaos	Multi-tenant platform, one tool for all clients
Enterprise IT teams	Shadow IT, no audit trail, cost overruns	RBAC + immutable audit + cost guardrails

Pricing Model (SaaS Tiers):

Tier	Price/month	Users	Projects	Features
Free	$0	3	2	AWS only, basic pipeline, community support
Starter	$99	10	10	AWS + security scans + cost estimates + email alerts
Pro	$299	50	50	AWS + Azure + custom templates + Slack alerts + audit export
Enterprise	Custom	Unlimited	Unlimited	All clouds + SLA + SSO + dedicated support + on-prem option

■ 15. ChatGPT / Cursor Prompts — Build Each Component

Copy-paste these prompts directly into ChatGPT or Cursor to generate each component. Always mention the exact tech versions for best results.

ChatGPT / Cursor Prompt — Spring Boot Project Setup

```
Create a Spring Boot 3.2 project with Java 17. Include: Spring Web, Spring Security 6, Spring Data JPA, PostgreSQL driver, Redis (Spring Cache), Flyway, Spring Actuator, Lombok, MapStruct, Bucket4j for rate limiting, and OpenAPI 3 (springdoc). Use Maven. Package structure: com.infrapilot.{api.v1, domain.model, domain.repository, domain.service, config, security, exception}. Add application.yml with profiles: dev and prod.
```

ChatGPT / Cursor Prompt — Vaadin 24 MainLayout with Navigation

```
Create a Vaadin 24 MainLayout.java using AppLayout. Add a left sidebar with navigation to these views: Dashboard, Deploy, Pipeline, Security, Cost, Monitor, Audit, Settings. Use Vaadin icons. Add a top header with logged-in user name and logout button. Enable @Push for WebSocket real-time updates. Use Java 17. Apply responsive CSS.
```

ChatGPT / Cursor Prompt — JWT Authentication — Spring Security 6

```
Implement JWT authentication in Spring Boot 3.2 with Spring Security 6. Create: JwtFilter.java (OncePerRequestFilter), JwtUtil.java (generate/validate tokens), AuthController.java (POST /auth/login, POST /auth/refresh), SecurityConfig.java (filter chain, RBAC with roles: ADMIN, DEVOPS, DEVELOPER, VIEWER). Passwords stored as BCrypt. No sessions — stateless. Return access token (15min TTL) and refresh token (7 days TTL) on login.
```

ChatGPT / Cursor Prompt — PostgreSQL Schema + Flyway Migrations

```
Write Flyway SQL migration files for a multi-tenant DevOps SaaS platform. Tables: organizations, users (with BCrypt password_hash, role enum), projects, deployments (with JSONB spec column, status enum), pipeline_runs, scan_results (with JSONB findings), cost_estimates, audit_events (append-only). Add PostgreSQL Row-Level Security (RLS) policies on all tables using org_id column. Use gen_random_uuid() for primary keys. Add proper indexes. PostgreSQL 15.
```

ChatGPT / Cursor Prompt — Jenkins Shared Library — Security Scan Step

```
Write a Jenkins Shared Library step called runSecurityScan.groovy. It should accept: scanType (secrets|iac|container), deploymentId, optional imageTag. For secrets: run gitleaks detect --source . --report-format json --report-path scan-secrets.json For iac: run checkov -d infrastructure/aws/templates --output json > scan-iac.json For container: run trivy image --format json --output scan-container.json {imageTag} After each scan: POST the JSON results to ${INFRAPILOT_API}/api/v1/scans/{deploymentId} via HTTP. If CRITICAL findings found in secrets or container scan: error('Critical findings') to abort.
```

ChatGPT / Cursor Prompt — Terraform VPC Module — AWS

```
Write a Terraform module for AWS VPC. Input variables: org_id, environment, cidr_block, aws_region, tags (map). Create: 1 VPC, 2 public subnets (different AZs), 2 private subnets (different AZs), 1 database subnet group (2 AZs), Internet Gateway, NAT Gateway (1, in first public subnet), Route tables with correct associations. Outputs: vpc_id, public_subnet_ids, private_subnet_ids, database_subnet_ids. Tag every resource with var.tags + Name tag. Terraform 1.6, AWS provider ~>5.0.
```

ChatGPT / Cursor Prompt — Infracost Jenkins Step

```
Write a Jenkins Shared Library step called runInfracost.groovy. Parameters: deploymentId, terraformPath (default: infrastructure/aws/templates). Steps: (1) Install infracost CLI if not present, (2) Run: infracost breakdown --path ${terraformPath} --format json --out-file infracost-output.json (3) Parse the JSON to extract totalMonthlyCost and resource breakdown (4) POST the result as JSON to
```

```
${INFRAPILOT_API}/api/v1/cost/{deploymentId} using httpRequest Jenkins plugin. (5) Print summary to console: 'Estimated cost: $X/month'. Do not fail the pipeline if Infracost fails — log warning and continue.
```

ChatGPT / Cursor Prompt — Vaadin Deployment Wizard View

```
Create DeployView.java in Vaadin 24. It should be a multi-step wizard using Vaadin tabs or a custom stepper: Step 1: Select project (ComboBox from API), cloud provider (Select: aws/azure/gcp), region (ComboBox filtered by provider), environment (dev/staging/prod). Step 2: Select template (fullstack-standard, container-standard), enter repo URL and branch. Step 3: Review summary + estimated cost (fetched from API) + Submit button. On submit: POST to /api/v1/deployments via RestTemplate or WebClient. After submit: navigate to PipelineView with the new deploymentId. Validate all fields before allowing next step. Java 17, Spring Boot 3.2.
```

ChatGPT / Cursor Prompt — Prometheus Custom Metrics in Spring Boot

```
Add custom Prometheus metrics to DeploymentService.java in Spring Boot 3.2 using Micrometer. Register these metrics: (1) Counter: infrapilot.deployments.total with tags: status, environment, provider (2) Timer: infrapilot.deployment.duration with tags: template, environment (3) Counter: infrapilot.scan.results.total with tags: tool, severity, status (4) Gauge: infrapilot.active.pipelines — count from DB query (5) Counter: infrapilot.cost.estimate.usd — increment by estimated monthly cost Show how to increment/record each metric in the service methods. Configure application.yml to expose /actuator/prometheus endpoint.
```

ChatGPT / Cursor Prompt — Docker Compose — Local Dev Environment

```
Write a docker-compose.yml for local development of InfraPilot. Services: (1) postgres:15 — port 5432, init with infrapilot database, health check (2) redis:7-alpine — port 6379 (3) vault:1.15 — dev mode, port 8200, VAULT_DEV_ROOT_TOKEN_ID=dev-token (4) prometheus:v2.48.0 — port 9090, with basic prometheus.yml scraping localhost:8080 (5) grafana:10.2.0 — port 3000, admin/admin, auto-provision datasource pointing to prometheus All services on a infrapilot-net bridge network. Add .env.example with all required environment variables. Add health checks for postgres and redis.
```

■ Summary — Your Build Checklist

Follow this order. Don't skip phases. Each checkbox = one working demo.

[ ] Phase 0: Repo setup, Docker Compose local env, Spring Boot + Vaadin skeleton running

[ ] Phase 1: Terraform modules deploy real AWS infrastructure end-to-end

[ ] Phase 2: Jenkins pipeline: checkout → scan → Infracost → plan → approve → apply → smoke test

[ ] Phase 3: Spring Boot API + SQS async deployment trigger working

[ ] Phase 4: Vaadin dashboard connected to live API — full deployment flow from UI

[ ] Phase 5: Vault secrets, PostgreSQL RLS tenant isolation, audit log all tested

[ ] Phase 6: Grafana dashboards live, alerts firing, DR drill completed

[ ] BONUS: Add GitHub Actions as alternative to Jenkins (shows DevOps breadth)

[ ] BONUS: Infracost PR comment bot — show cost estimate on every GitHub PR

[ ] BONUS: Slack bot integration — deployment status notifications to team channel

GitHub mein README.md mein yeh cheezein zaroor add karo: Architecture diagram (draw.io), 5-minute demo video link, 'Quick Start' section, live demo link if possible. Recruiter pehle README dekhta hai. README strong = project strong.

InfraPilot | Developer Build Guide v1.0 | Senior Dev Review Edition

Stack: Java 17 + Spring Boot 3 + Vaadin 24 + PostgreSQL + Redis + Jenkins + Terraform + Docker + AWS + Prometheus + Grafana